Jupyter Notebook
The Complete Guide

Setup • Interface • Shortcuts • Magic Commands

Best-Practice Code Suggestions • Tips & Troubleshooting

A practical reference for beginners through advanced users

Table of Contents

1. What Is Jupyter Notebook?
2. Installation & Launching
3. Interface Overview
4. Cells, Modes & the Kernel
5. Essential Keyboard Shortcuts
6. Magic Commands Reference
7. Markdown Cheat Sheet for Notebooks
8. Code Suggestions & Best Practices
9. Handy Code Snippets
10. Extensions & Productivity Tools
11. Exporting & Sharing Notebooks
12. Troubleshooting Common Issues
13. Quick-Reference Cheat Sheet

1. What Is Jupyter Notebook?

Jupyter Notebook is an open-source, web-based interactive computing environment. It lets you combine live code, equations, visualizations, and narrative text (Markdown) in a single document called a **notebook** (.ipynb file). It is widely used for data analysis, machine learning, scientific computing, teaching, and reproducible research.

Why use it?

- **Interactive execution** — run code cell by cell and immediately see results.
- **Rich output** — tables, plots, images, HTML, and LaTeX render inline.
- **Narrative + code together** — document your reasoning next to the code that implements it.
- **Language-agnostic** — supports Python, R, Julia, and 100+ languages via kernels.
- **Shareable** — export to HTML, PDF, slides, or share the .ipynb file directly.

Jupyter Notebook vs. JupyterLab

JupyterLab is the next-generation interface for Jupyter — it includes the classic Notebook experience plus a file browser, multiple panes, a terminal, and extension support in one window. Classic Notebook (and Notebook 7, which is built on JupyterLab components) is simpler and single-document focused. Shortcuts and concepts in this guide apply to both, with minor menu differences noted where relevant.

2. Installation & Launching

Option A — Using pip

```
pip install notebook
jupyter notebook          # launches classic Notebook interface
```

Option B — Using JupyterLab

```
pip install jupyterlab
jupyter lab                # launches the JupyterLab interface
```

Option C — Using Anaconda (recommended for data science)

Anaconda bundles Python, Jupyter, and hundreds of scientific packages (NumPy, pandas, Matplotlib, scikit-learn) in one installer. After installing Anaconda, launch Jupyter from the Anaconda Navigator or from a terminal:

```
conda install -c conda-forge notebook
jupyter notebook
```

Creating an isolated environment (best practice)

```
# with venv
python -m venv myenv
source myenv/bin/activate     # Windows: myenv\Scripts\activate
pip install notebook ipykernel
python -m ipykernel install --user --name=myenv

# with conda
conda create -n myenv python=3.11 notebook
conda activate myenv
```

■ Tip

Always work inside a virtual environment. It keeps project dependencies isolated and avoids version conflicts between projects.

Launching a Notebook

```
cd path/to/your/project
jupyter notebook          # opens in your default browser at localhost:8888
# or
jupyter lab
```

You can also specify a port or disable the auto-open browser: `jupyter notebook --port 8890 --no-browser`.

3. Interface Overview

- **Menu bar** — File, Edit, View, Insert, Cell/Run, Kernel, Help.
- **Toolbar** — quick buttons: save, add cell, cut/copy/paste cell, run, stop, restart kernel, cell-type dropdown.
- **Notebook name** — click the title at the top to rename the file.
- **Cell area** — the main body where you write and run code or Markdown.
- **Kernel indicator** — top-right circle: hollow = idle, filled = busy executing.
- **File browser** (JupyterLab) — left sidebar to navigate folders and open files.

The Kernel

The kernel is the computational engine (e.g., Python process) that executes the code in your cells and keeps variables in memory between runs. Restarting the kernel clears all variables; re-running cells from top to bottom rebuilds state cleanly.

4. Cells, Modes & the Kernel

Cell Types

- **Code cell** — contains executable code in the kernel's language.
- **Markdown cell** — contains formatted text, headings, lists, images, LaTeX math.
- **Raw cell** — plain text passed through unmodified during notebook conversion (nbconvert).

Two Input Modes

Jupyter has two distinct keyboard modes, and knowing which one you're in explains almost every shortcut in this guide.

- **Command Mode** (blue left border) — press **`Esc`** to enter. The cell is selected but not editable; keystrokes are notebook-level commands (add cell, delete cell, change cell type, etc.).
- **Edit Mode** (green left border) — press **`Enter`** (or click inside a cell) to enter. You are typing directly into the cell as you would in a text editor.

> **■ Tip**
> If a shortcut doesn't seem to work, check the border color. Most single-letter shortcuts (like 'a', 'b', 'dd') only work in Command Mode — press Esc first.

5. Essential Keyboard Shortcuts

Running Cells (works in both modes)

Shortcut	Action
`Shift + Enter`	Run the current cell and select/create the next cell
`Ctrl + Enter`	Run the current cell and keep selection on it
`Alt + Enter`	Run the current cell and insert a new cell below
`Ctrl + Shift + Enter`	Run all cells above and including this one (varies by version)

Command Mode — press Esc first

Shortcut	Action
`A`	Insert a new cell Above
`B`	Insert a new cell Below
`D, D`	Delete the selected cell (press D twice)
`Z`	Undo cell deletion
`M`	Change cell type to Markdown
`Y`	Change cell type to Code
`R`	Change cell type to Raw
`C`	Copy selected cell(s)
`X`	Cut selected cell(s)
`V`	Paste cell(s) below
`Shift + V`	Paste cell(s) above
`Shift + M`	Merge selected cells
`Ctrl + Shift + -`	Split cell at cursor (in Edit Mode)
`↑ / K`	Select cell above
`↓ / J`	Select cell below
`Shift + ↑ / ↓`	Extend selection to multiple cells
`L`	Toggle line numbers in the cell

Shortcut	Action
`Shift + L`	Toggle line numbers in all cells
`O`	Toggle output of selected cell
`I, I`	Interrupt the kernel (press I twice)
`0, 0`	Restart the kernel (press 0 twice)
`F`	Find and replace
`H`	Open the keyboard shortcuts help dialog
`S / Ctrl + S`	Save notebook and checkpoint
`Shift + Space`	Scroll notebook up

Edit Mode — press Enter to activate

Shortcut	Action
`Tab`	Code completion / indent
`Shift + Tab`	Show docstring / function signature (tooltip)
`Ctrl + /`	Toggle comment on line(s)
`Ctrl + A`	Select all text in the cell
`Ctrl + Z`	Undo
`Ctrl + Shift + Z / Ctrl + Y`	Redo
`Ctrl + Home`	Go to start of cell
`Ctrl + End`	Go to end of cell
`Ctrl + ]`	Indent selected lines
`Ctrl + [`	Dedent selected lines
`Esc`	Switch to Command Mode
`Ctrl + Shift + P`	Open the command palette (search any action)

JupyterLab-Specific

Shortcut	Action
`Ctrl + Shift + C`	Open the command palette
`Ctrl + B`	Toggle the left sidebar (file browser)

Shortcut	Action
`Ctrl + Shift + D`	Toggle single-document mode
`Ctrl + K, then V`	Open Markdown preview in a split pane

■ Tip

Press H in Command Mode at any time to see the full, version-specific shortcut list built into your Jupyter installation — shortcuts can vary slightly between versions.

6. Magic Commands Reference

Magic commands are special IPython commands prefixed with % (line magic, affects one line) or %% (cell magic, affects the whole cell). They are not regular Python syntax but shortcuts built into the kernel.

Line Magics

Shortcut	Action
`%time / %timeit`	Time execution of one statement (timeit averages over multiple runs)
`%who / %whos`	List variables in the current namespace (whos = with details)
`%reset`	Clear all variables from the namespace
`%pwd`	Print current working directory
`%cd path`	Change the working directory
`%ls`	List files in the current directory
`%run script.py`	Run an external Python script inside the notebook
`%load script.py`	Load the contents of a script into the current cell
`%matplotlib inline`	Render Matplotlib plots directly below the cell
`%env`	List or set environment variables
`%history`	Show previously executed commands
`%pip install pkg`	Install a package into the current kernel's environment
`%lsmagic`	List all available magic commands

Cell Magics

Shortcut	Action
`%%time`	Time execution of the entire cell
`%%writefile file.py`	Write the cell's contents to a file
`%%bash`	Run the cell as a Bash script
`%%html`	Render the cell contents as raw HTML
`%%latex`	Render the cell contents as LaTeX
`%%capture`	Suppress or capture a cell's stdout/stderr output
`%%timeit`	Benchmark the entire cell, averaged over several runs

```
# Example usage
%matplotlib inline
import matplotlib.pyplot as plt

%%timeit
sum(range(1_000_000))

%%writefile utils.py
def greet(name):
    return f"Hello, {name}!"
```

7. Markdown Cheat Sheet for Notebooks

Switch a cell to Markdown with **M** (Command Mode), then run it with Shift+Enter to render.

Markdown	Result
`# Heading 1`	Largest heading
`## Heading 2`	Second-level heading
`**bold** / *italic*`	Bold and italic text
`` `inline code` ``	Inline monospace code
`- item / 1. item`	Bullet or numbered list
`[text](url)`	Hyperlink
`![alt](image.png)`	Embed an image
`> quote`	Blockquote
`---`	Horizontal rule
`$e=mc^2$`	Inline LaTeX math (via MathJax)
`$$\int_a^b f(x)dx$$`	Block/display LaTeX math
```` ```python ... ``` ````	Fenced, syntax-highlighted code block

8. Code Suggestions & Best Practices

Structuring a Notebook

- Start with a Markdown title cell and a short description of the notebook's purpose.
- Group imports in the first code cell so dependencies are visible at a glance.
- Use Markdown headings (##, ###) to break the notebook into logical sections (Data Loading, Cleaning, Modeling, Evaluation, Conclusions).
- Keep each cell focused on one task — easier to debug and re-run individually.
- Restart the kernel and 'Run All' before sharing, to confirm the notebook runs top-to-bottom without hidden state.

Recommended Import Block

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
pd.set_option('display.max_columns', 100)
plt.style.use('seaborn-v0_8')
```

Avoid Hidden State Bugs

Because cells can run out of order, a notebook can behave inconsistently between your session and a fresh run. Guard against this:

- Use **Kernel** → **Restart & Run All** before finishing a session.
- Avoid re-defining the same variable name for different purposes across cells.
- Number or clearly label experimental cells you plan to delete later.

Autoreload for Local Modules

When editing a helper .py file alongside your notebook, autoreload picks up changes without restarting the kernel:

```
%load_ext autoreload
%autoreload 2

from my_module import my_function   # edits to my_module.py apply automatically
```

Reproducibility

- Set random seeds explicitly: `np.random.seed(42)`.
- Pin package versions with `pip freeze > requirements.txt`.
- Avoid absolute file paths; use relative paths or `pathlib.Path`.
- Store large datasets outside the notebook and load them explicitly, rather than hardcoding pasted data.

Performance Tips

- Profile slow cells with `%%timeit` or `%prun` before optimizing.
- Use vectorized NumPy/pandas operations instead of Python for-loops over rows.
- For very large outputs, display a sample (`df.head()`) instead of the full DataFrame.
- Clear unused large variables with `del var_name` to free memory in long sessions.

> **■ Tip**
>
> Use the Tab key for autocompletion and Shift+Tab to inspect a function's signature and docstring without leaving the notebook.

9. Handy Code Snippets

Display all outputs in a cell (not just the last)

```
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"
```

Suppress warnings cleanly

```
import warnings
warnings.filterwarnings('ignore')
```

Progress bars for loops

```
from tqdm.notebook import tqdm

for i in tqdm(range(1000)):
    pass
```

Interactive widgets

```
import ipywidgets as widgets
from IPython.display import display

slider = widgets.IntSlider(min=0, max=100, value=50)
display(slider)
```

Rendering wide DataFrames and images nicely

```
pd.set_option('display.max_rows', 200)
pd.set_option('display.max_colwidth', None)

from IPython.display import Image, display
display(Image(filename='chart.png', width=400))
```

Saving figures at publication quality

```
plt.savefig('figure.png', dpi=300, bbox_inches='tight')
```

10. Extensions & Productivity Tools

Tool	What it does
`jupyter_contrib_nbextensions`	Classic Notebook add-ons: spell checker, code folding, collapsible headings, ExecuteTime
`jupyterlab-lsp + jedi-language-server`	Real autocompletion, go-to-definition, and linting in JupyterLab
`nbdime`	Human-readable diffing and merging of notebooks in version control
`jupytext`	Pairs .ipynb files with plain .py/.md files for cleaner git diffs
`ipywidgets`	Interactive sliders, buttons, and dropdowns inside a notebook
`nbconvert`	Command-line conversion of notebooks to HTML, PDF, scripts, slides
`black / nb_black`	Auto-format code cells on run
`Variable Inspector`	Sidebar panel listing all current variables and their values

```
pip install jupyter_contrib_nbextensions
jupyter contrib nbextension install --user

pip install jupyterlab-lsp python-lsp-server
```

11. Exporting & Sharing Notebooks

From the UI

File → Download as / Export Notebook As → choose HTML, PDF, Markdown, Python script, or Reveal.js slides.

From the command line (nbconvert)

```
jupyter nbconvert --to html notebook.ipynb
jupyter nbconvert --to pdf notebook.ipynb
jupyter nbconvert --to script notebook.ipynb     # extract as .py
jupyter nbconvert --to slides notebook.ipynb --post serve
```

■ Tip

PDF export via nbconvert requires a LaTeX installation (e.g., TeX Live or MiKTeX). If that's unavailable, export to HTML first and print to PDF from the browser.

Sharing options

- **GitHub** — renders .ipynb files natively in the repo browser.
- **nbviewer.org** — paste a link to any public .ipynb for clean rendering.
- **Google Colab / Binder** — share a live, runnable version of your notebook.
- **Voilà** — turns a notebook into a standalone interactive web app, hiding the code.

12. Troubleshooting Common Issues

Issue	Fix
`Kernel keeps dying / restarting`	Check for infinite loops, memory-heavy operations, or a corrupted package; watch the terminal log for the crash reason.
`'Module not found' despite pip install`	The notebook may be using a different kernel/environment; run %pip install pkg inside the notebook or select the correct kernel from Kernel → Change Kernel.
`Notebook won't open / shows raw JSON`	The .ipynb file may be corrupted; try Cell → All Output → Clear, or open it with nbformat to repair, or restore from a checkpoint.
`Plots not showing`	Add %matplotlib inline at the top of the notebook (classic) — usually automatic in modern versions.
`Cell runs forever`	Interrupt with the stop button or press I, I in Command Mode; if unresponsive, restart the kernel.
`Out-of-order execution confusion`	Use Kernel → Restart & Run All to verify the notebook is internally consistent.
`Port already in use`	Launch with jupyter notebook --port 8891 to use a different port.
`Autocomplete (Tab) not working`	Ensure the kernel is idle (not busy); install jupyterlab-lsp for richer completions.

13. Quick-Reference Cheat Sheet

Most-used shortcuts at a glance

Shortcut	Action
`Shift + Enter`	Run cell, move to next
`Esc then A / B`	Insert cell above / below
`Esc then D D`	Delete cell
`Esc then M / Y`	Markdown / Code cell
`Tab`	Autocomplete
`Shift + Tab`	Docstring / signature
`Ctrl + /`	Comment line
`Esc then 0 0`	Restart kernel
`Esc then I I`	Interrupt kernel
`Ctrl + S`	Save notebook

Most-used magics

Shortcut	Action
`%matplotlib inline`	Inline plots
`%timeit`	Benchmark a line
`%%time`	Time a whole cell
`%who`	List variables
`%run file.py`	Run a script
`%pip install pkg`	Install a package

You're ready to work efficiently in Jupyter Notebook — from setup, through daily keyboard-driven workflow, to exporting polished results. Keep this guide handy as a reference while you build muscle memory for the shortcuts.